Course Title: Object-Oriented Programming using Java
Project Name: Hotel Reservation Management System
Submitted By: Medimi Nishit Kumar
Roll Number: 63

Enrollment Number: 12024052017065
Instructor: Dr.Soumadip Biswas
Academic Session: Spring 2026

EXECUTIVE SUMMARY

This project presents a **console-based Hotel Reservation Management System** that facilitates efficient handling of room bookings, guest records, and transaction tracking. The application is designed using Java and adheres strictly to Object-Oriented Programming principles.

The system follows a **Controller–Store architecture**, where:

- Controllers handle business logic and validation
- Stores manage data persistence using CSV files

This separation ensures better maintainability, scalability, and modularity.

PROBLEM ANALYSIS

In traditional hotel systems, manual booking processes lead to:

- Inefficient room allocation
- Data inconsistency
- Difficulty in managing guest information
- Errors in billing and booking duration

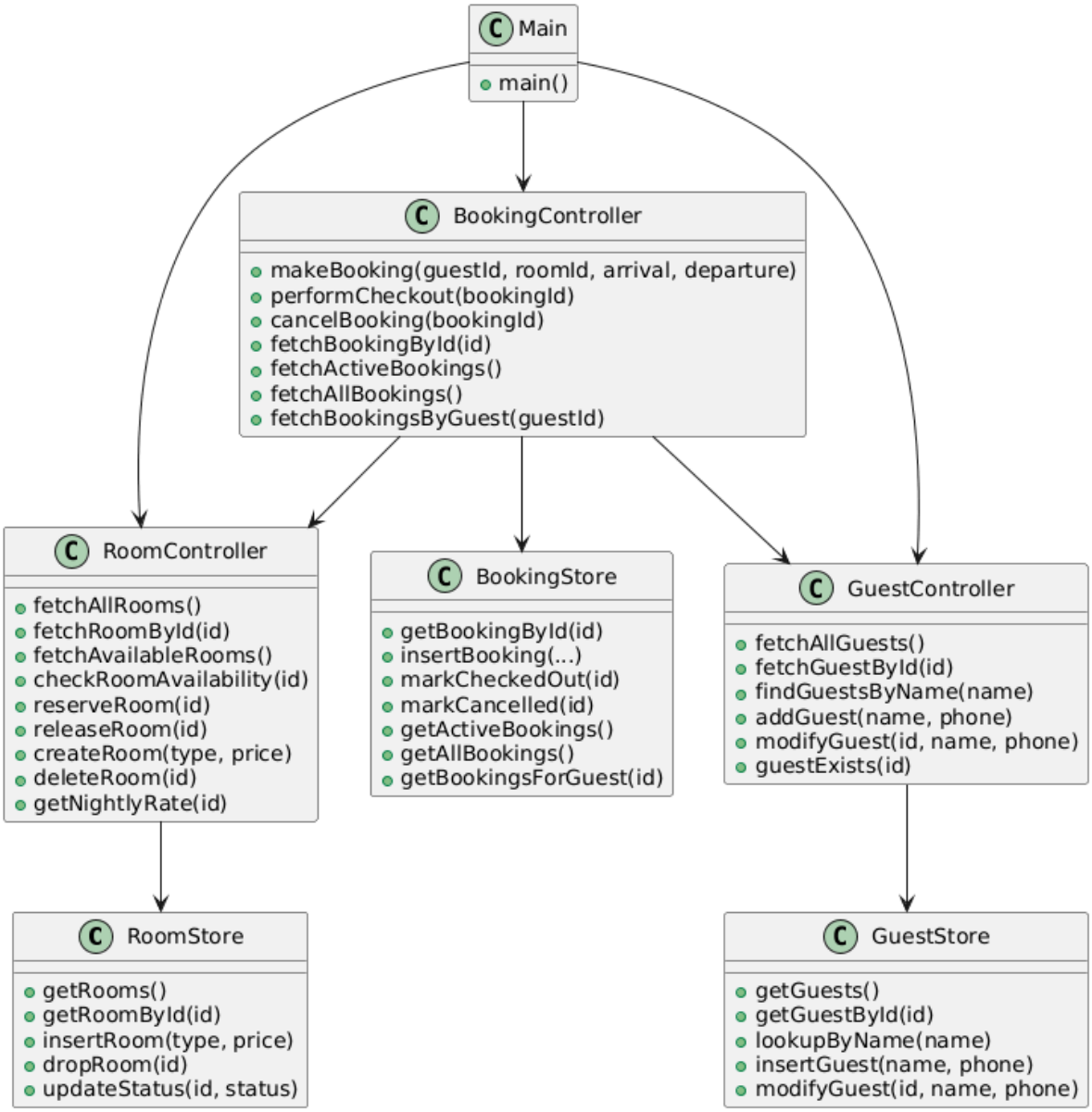
The proposed system aims to automate these processes with:

- Real-time room availability tracking
- Structured guest management
- Accurate booking lifecycle handling
- Reliable file-based storage

The system is particularly suitable for small-scale hotels that require a **low-cost, database-free solution**.

SYSTEM DESIGN AND ARCHITECTURE

UML CLASS DIAGRAM



ARCHITECTURAL DESIGN

The system adopts a **three-layer structure**:

Layer	Responsibility
Presentation Layer	Handles user interaction (`Main`)
Controller Layer	Business logic & validation
Data Layer (Store)	File handling and persistence

Example:

MAJOR COMPONENTS

RoomController

- Retrieves and manages room data
- Checks availability
- Updates room status

GuestController

- Handles guest registration and updates
- Performs search operations

BookingController

- Manages booking lifecycle
- Validates guest and room
- Calculates total cost

Store Classes

- Responsible for reading/writing CSV files
- Maintain in-memory maps for fast access

4.4 PACKAGE STRUCTURE

```
controllers/
 ├── BookingController.java
 ├── GuestController.java
 └── RoomController.java

store/
 ├── BookingStore.java
 ├── GuestStore.java
 └── RoomStore.java

Main.java
```

4.5 PROCESS FLOW

```
User Input
   ↓
Main Menu
   ↓
Controller Layer
   ↓
Store Layer (CSV)
   ↓
Response Output
```

Booking Example Flow:

```
BookingController.makeBooking()
→ Validate Guest
→ Check Room Availability
→ Calculate Cost
→ Store Booking
→ Update Room Status
```

APPLICATION OF OOP PRINCIPLES

Encapsulation

Implementation: Store classes
Purpose: Protect internal data structures

```
private final LinkedHashMap<Integer, Map<String, String>> bookingTable;
```

Explanation:
Data is restricted and accessed through defined methods, ensuring integrity.

Abstraction

Implementation: Controllers
Purpose: Hide storage and processing complexity

```
public String makeBooking(int guestId, int roomId, String arrivalDate,
String departureDate)
```

Explanation:
User interacts with simple methods without understanding underlying logic.

Composition

Implementation: BookingController

```
private final RoomController roomCtrl;
private final GuestController guestCtrl;
```

Explanation:
Complex operations are built using multiple components

Polymorphism

Implementation: Flexible method behavior

```
public Map<String, String> fetchBookingById(int bookingId)
```

Explanation:
Same method structure supports different data retrieval operations.

IMPLEMENTATION HIGHLIGHTS

Booking Validation Logic

```
if (!guestCtrl.guestExists(guestId))
    return "Error: No guest found";
```

Justification:
Ensures only valid users can book rooms.

Date Validation

```
if (!departure.isAfter(arrival))
    return "Error: Departure must be after arrival";
```

Justification:
Prevents invalid booking durations.

Cost Calculation

```
long totalNights = ChronoUnit.DAYS.between(arrival, departure);
int amountDue = totalNights * nightlyRate;
```

Justification:
Ensures accurate billing based on duration.

Room Status Update

```
roomCtrl.reserveRoom(roomId);
```

Justification:
Maintains real-time availability.

TESTING AND ERROR MANAGEMENT

TEST CASES

Scenario	Expected Result
Valid booking	Success
Invalid guest	Error
Invalid date	Rejection
Checkout	Room freed

EDGE CONDITIONS

- Incorrect date format
- Booking overlap
- Non-existent IDs
- Empty data files

ERROR HANDLING

```
catch (DateTimeParseException e) {
    return "Invalid date format";
}
```

Handled cases:

- File I/O issues
- Parsing errors
- Invalid user input

VERSION CONTROL STRATEGY

8.1 COMMIT BREAKDOWN

- Commit 1 → Store layer implementation
- Commit 2 → Controller layer
- Commit 3 → Main interface
- Commit 4 → Integration
- Commit 5 → Testing & debugging

8.2 DEVELOPMENT FLOW

1. Designed data storage
2. Implemented controllers
3. Integrated UI
4. Tested workflows
5. Optimized logic

9. CONCLUSION

The system effectively demonstrates the application of Object-Oriented Programming in building a real-world system. The Controller–Store design improves modularity and ensures a clear separation between business logic and data handling.

The system is efficient, reliable, and suitable for small-scale hotel management.

10. FUTURE IMPROVEMENTS

- Database integration (SQL)
- GUI-based interface
- Online reservation system
- Payment gateway integration
- Multi-user access
- Advanced analytics dashboard